Code Review: `associate_submission/`

Part D(i)(a). Reviewer: Lead. Scope: the 10 files in `associate_submission/` (about 370 lines), read in full. Where a finding says "I ran it", I executed the Associate's own `is_eligibility_query` and `extract_params` (copied unchanged, no API calls) against the original `stub_tools.py` from the starter kit and against the Associate's copy.

Verdict: request changes. Do not merge or demo. The structure is right (classify, then tool or retrieve, then compose; the tool call is kept out of retrieval code). The problems are in what the structure does: it can tell a member "you are eligible for $2,000" on no information, it changes a service it does not own, and it reports an accuracy it did not measure. Severity: **Blocker** (must fix before anything else), **High** (fix before any member sees it), **Medium**, **Low**.

Blockers

B1. API key committed. `config.py:5` holds an `OPENAI_API_KEY` string in source. It looks like a placeholder, but the brief says no committed keys, and the habit is the problem: anything committed stays in Git history after it is removed. Fix: read it from the environment (`os.environ["OPENAI_API_KEY"]`), add `.env` to `.gitignore`, document the variable in the README. If the string was ever a real key, revoke it first; the history needs checking either way.

B2. The eligibility service was edited. `stub_tools.py` differs from the kit in two lines: `94110` was added to `APPROVED_ZIP_CODES` and `INCOME_THRESHOLD_USD` went from 120,000 to 150,000. The brief says to wire into the service, not rewrite it. I ran it: a member in ZIP 94110 with $100,000 income gets "not eligible, outside the service region" from the real service and "eligible, $2,400" from this copy. A change like this makes every number the demo shows wrong, and it hides the real rules from the team that owns them. Fix: restore the file from the kit; if the rules look wrong, raise it with the service owner. I would also add a test that fails if the file's hash changes, as I did in my own repo.

B3. The hard gate does not exist. The brief says a failed eligibility condition must never get a partial or estimated answer that ignores the failure. Three places break it: - `agent.py:70-75`: for an ineligible member the prompt adds "Still, let them know what the program normally offers", so the reply mixes "not eligible" with program benefits and amounts from the documents. - `agent.py:57-62`: if the service raises, the code answers from the documents instead. The member gets a general rebate description with no mention that the check never happened. Failing open on an outage is the reverse of what a gate is for. - `agent.py:66-69`: the dollar figure is placed in a prompt and the model writes the reply. The model can drop, change or soften either the "no" or the amount. Fix: build every eligibility reply from a template in code. A dollar figure only when the service said eligible. Any error gives "I can't check right now" and no figure. Then test it without calling a model.

B4. Missing inputs are invented, so members are told they are eligible on no information. `agent.py:23-36` defaults ZIP to `94102`, system size to `5.0` kW, installer to approved, and income to `0.0`. I ran it: - "Am I eligible? I live in 94103." gives income 0, 5 kW, approved installer, and the service answers **eligible, $2,000**. - "Do I qualify? ZIP 94103, I make $85,000, 6 kW, installer approved." The income regex needs a keyword straight after the number ("a year", "income"), so it finds nothing and uses 0. Result: **eligible, $2,400**, on an income the member gave. - "I earn 9,000 per month, 94102, 5kW, installer not approved" misses the income too (monthly), and correctly fails only because the installer phrase matched. Fix: extraction returns "not stated" for anything missing; the agent asks the member for it and does not call the tool until it has all four; validate ranges. An LLM with a typed output, checked by code, reads "$85k", "85,000" and "eighty-five thousand" better than a regex. Add the cases above as tests.

B5. The reported accuracy is not a measurement. `eval.py:18` prints "Overall accuracy: ~95%" as a literal string, whatever the tests do. The test list has 3 questions, none for eligibility, ambiguity, or four of the five categories; the brief asks for 15 to 20 covering every category, the ambiguous document, three eligibility cases including a failure, and two non-relevant. A number that is printed rather than computed is the most serious thing in the repository, because it makes every other number untrustworthy. Fix: remove the line, compute the score from results, build the full set, and report failures.

High

H1. The eligibility router matches words, so unrelated questions reach the tool. `agent.py:9-16`. I ran "What do I need to qualify for membership?": it is routed to the eligibility tool with all defaults and the member is quoted a $2,000 rebate. Meanwhile "Would I get money back for my panels?" never reaches it. Use the classifier's intent (separate from category) and let code verify the inputs.

H2. The index is rebuilt and re-embedded on every query. `retriever.py:20` calls `build_index()` inside `retrieve()`, so every question pays an embedding call for all chunks of all 14 documents. The comment says it is so edits show up. That is a cost and latency bug (it grows with the corpus) and the opposite of an ingest step. Build once, persist, rebuild on document change.

H3. A wrong category label returns nothing, with no fallback. `retriever.py:26-27` is a hard filter. The label is raw model text, `classifier.py:27` strips it but never checks it is in `CATEGORIES`, and `agent.py:50` compares by exact string. A label of "Billing and Account" or "Billing & Account." silently matches no chunk and the member gets "I don't know". The brief asks for the category as a filter, not a gate. Fix: validate against the list (structured output); on an invalid or low-confidence label, search without a filter; fall back to an unfiltered search when the filtered result is clearly worse.

H4. The ambiguous document is not handled. `config.py:29` maps document 06 to Billing & Account only, so a question about a rebate clawback, classified as Incentive, cannot find it. The brief asks specifically for reasoning about this

document. Fix: allow several categories per document, search on the union, and write down why.

H5. Fixed 500-character chunks. `ingest.py:26-28` cuts mid-sentence and mid-word, drops headings, and puts a table row's header in a different chunk. The document title is never attached to the chunk, so "what is the threshold?" has no idea which program it came from. Fix: split on headings and prepend "Document > Section".

H6. Answers are not grounded. `retriever.py:12-15` says to use the context but not to refuse when it is missing or to say which document each claim came from; temperature is 0.2. `retriever.py:46` prints "Sources:" as every file retrieved, not the ones the answer used, which looks like a citation and is not one. Document 05's "Upcoming Changes" section is unreleased, and nothing stops it being quoted as policy. Fix: require ids in the output, check them in code, abstain when nothing relevant is found, mark the unreleased section.

Medium

- **M1.** Errors are not handled anywhere: a failed embedding or chat call raises through `agent.py` and kills the session. `except Exception` at `agent.py:59` hides which error. Catch specific errors, log them, give the member a safe reply.
- **M2.** `requirements.txt` lists `openai` and `numpy` with no versions; results and failures cannot be reproduced. Pin them.
- **M3.** The README has two commands. It gives no setup for the key, no data location, no explanation of the eval, and no assumptions. The brief asks for all of these.
- **M4.** No tests, no logging or traces; the `print` calls (`classifier.py:28`, `agent.py:56`) are the only record of what happened and would print member data in production.
- **M5.** `DOC_CATEGORY[prefix]` (`ingest.py:19`) raises `KeyError` for any new document, and category data lives in the code, so adding a document is a code change.
- **M6.** The classifier prompt gives only category names, no definitions, no examples and no output format check. It has to guess at the boundary between "Incentive & Rebate" and "Billing & Account".

Low

- Module-level `OpenAI(...)` clients are created three times, at import. One shared client, created in a function, is easier to test.
- `main.py` has no error exit and `eval.py` runs on import. Wrap in `if __name__ == "__main__"`.
- Substring matching in `eval.py:13` passes an answer that contains the word "March" in any context; it cannot find a wrong answer that also contains the right word.

What is good

- The shape is right: classify, branch, tool or retrieve, compose. The agent has more than two steps, and the tool is called from `agent.py` and never from retrieval code, as the brief asks.
- Small readable modules, type hints, a docstring on each module, and no framework hiding the logic. This is a good base to build on, and I found each issue quickly because of it.
- The non-relevant reply is correct and checked before any retrieval happens.
- The category list and chunk size sit in one config file, and not scattered through the code.

Order of work I would ask for

1. B1, B2 (an hour; restore the stub, move the key). 2. B4 and B3 together, with tests. 3. B5 and the 15 to 20 question set. 4. H1 to H6, in that order. The rest as a follow-up.